

1. Suppose your training examples are sentences (sequences of words). Which of the following refers to the j^{th} word in the i^{th} training example?

1 / 1 point

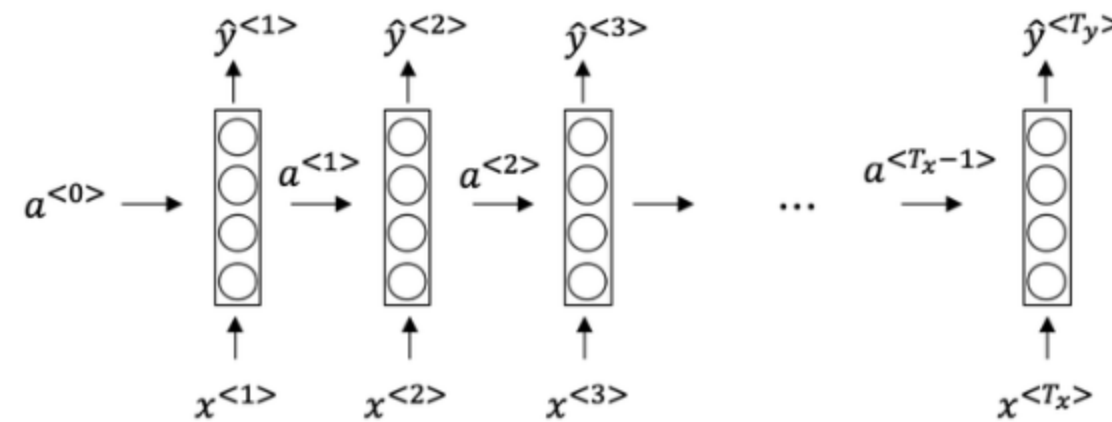
- ☒ $x^{(i)<j>}$
- ☐ $x^{<i>(j)}$
- ☐ $x^{(j)<i>}$
- ☐ $x^{<j>(i)}$

✓ Nice work

We index into the i^{th} row first to get the i^{th} training example (represented by parentheses), then the j^{th} column to get the j^{th} word (represented by the brackets).

2. Consider this RNN:

1 / 1 point



This specific type of architecture is appropriate when:

- ☒ $T_x = T_y$
- ☐ $T_x < T_y$
- ☐ $T_x > T_y$
- ☐ $T_x = 1$

✓ Nice work

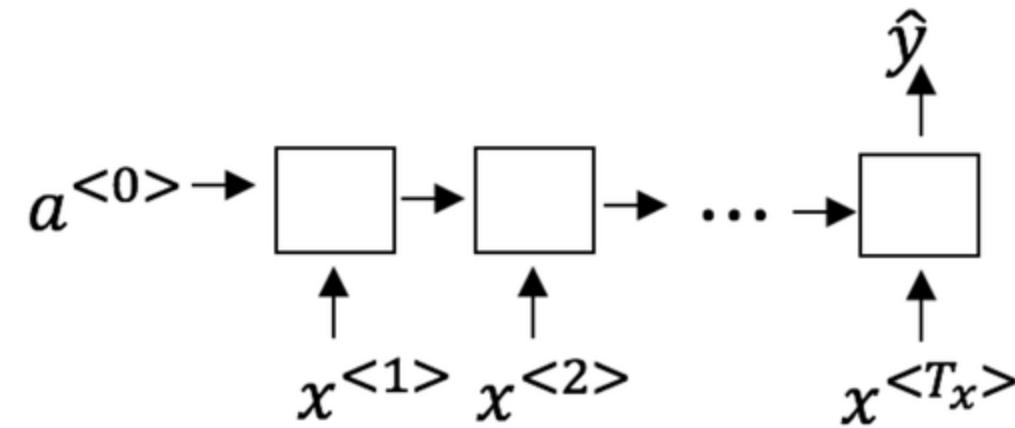
It is appropriate when every input should have an output.

✔ Nice work

It is appropriate when every input should have an output.

3. To which of these tasks would you apply a many-to-one RNN architecture?

1 / 1 point



☐ Image classification (input an image and output a label)

☒ Music genre recognition

✔ Nice work

This is an example of many-to-one architecture.

☒ Language recognition from speech (input an audio clip and output a label indicating the language being spoken)

✔ Nice work

This is an example of many-to-one architecture.

☐ Speech recognition (input an audio clip and output a transcript)

4. Using this as the training model below, answer the following:

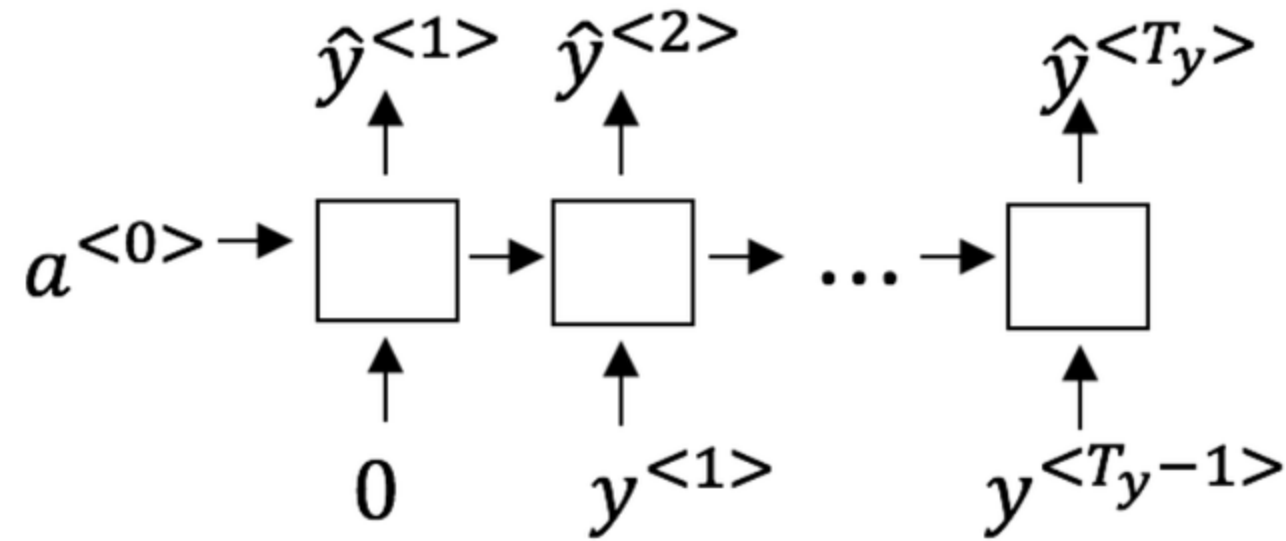
1 / 1 point

$\hat{v}^{<1>}$ $\hat{v}^{<2>}$ $\hat{v}^{<T_y>}$

☐ Speech recognition (input an audio clip and output a transcript)

4. Using this as the training model below, answer the following:

1 / 1 point



True/False: At the t^{th} time step the RNN is estimating $P(y^{<t>} | y^{<1>}, y^{<2>}, \dots, y^{<t-1>})$

☒ True

☐ False

✔ Nice work

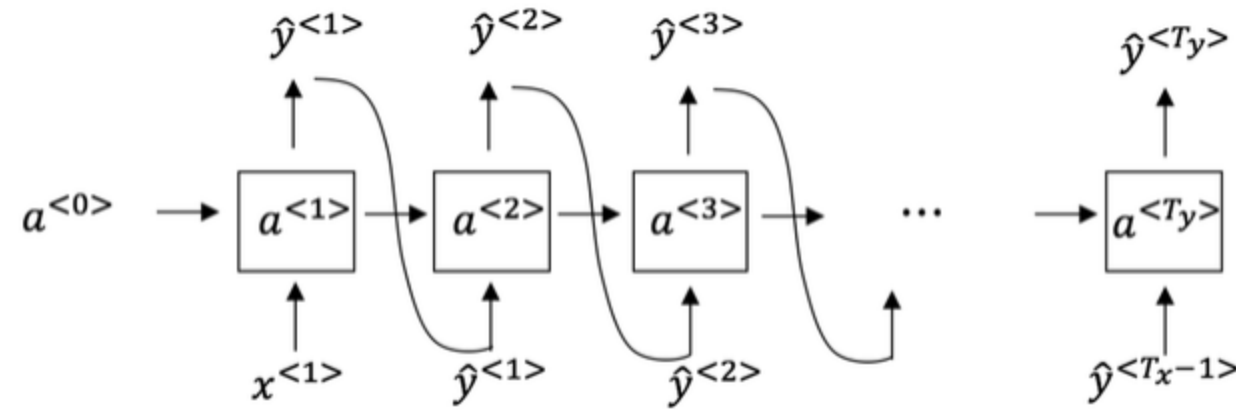
Yes, in a training model we try to predict the next step based on knowledge of all prior steps.

5. You have finished training a language model RNN and are using it to sample random sentences, as follows:

1 / 1 point

5. You have finished training a language model RNN and are using it to sample random sentences, as follows:

1 / 1 point



True/False: In this sample sentence, step t uses the probabilities output by the RNN to pick the highest probability word for that time-step. Then it passes the ground-truth word from the training set to the next time-step.

- ☒ False
- ☐ True

✔ Nice work

The probabilities output by the RNN are not used to pick the highest probability word and the ground-truth word from the training set is not the input to the next time-step.

6. True/False: If you are training an RNN model, and find that your weights and activations are all taking on the value of NaN ("Not a Number") then you have an exploding gradient problem.

1 / 1 point

- ☒ True
- ☐ False

✔ Nice work

Correct! Exploding gradients happen when large error gradients accumulate and result in very large updates to the NN model weights during training. These weights can become too large and cause an overflow, identified as NaN.

7. Suppose you are training an LSTM. You have an 80000 word vocabulary, and are using an LSTM with 800-dimensional activations $a^{<t>}$. What is the dimension of Γ_u at each time step?

1 / 1 point

- ☐ 100
- ☐ 8
- ☒ 800
- ☐ 80000

✔ Nice work

Correct, Γ_u is a vector of dimension equal to the number of hidden units in the LSTM.

8. Consider the update equations for the GRU:

1 / 1 point

$$\tilde{c}^{<t>} = \tanh(W_c [\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u [c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r [c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

Alice proposes a simplification where the update gate is removed ($\Gamma_u = 0$).

Betty proposes a simplification where the relevance gate is removed ($\Gamma_r = 1$).

Which model is more robust to the vanishing gradient problem while remaining functional for sequence learning?

- ☒ Betty's model (removing Γ_r), because it retains the update gate Γ_u , allowing the model to decide when to update or keep the memory state.
- ☐ Alice's model (removing Γ_u), because it guarantees that the memory cell state is never overwritten by

8. Consider the update equations for the GRU:

1 / 1 point

$$\tilde{c}^{(t)} = \tanh(W_c [\Gamma_r * c^{(t-1)}, x^{(t)}] + b_c)$$

$$\Gamma_u = \sigma(W_u [c^{(t-1)}, x^{(t)}] + b_u)$$

$$\Gamma_r = \sigma(W_r [c^{(t-1)}, x^{(t)}] + b_r)$$

$$c^{(t)} = \Gamma_u * \tilde{c}^{(t)} + (1 - \Gamma_u) * c^{(t-1)}$$

$$a^{(t)} = c^{(t)}$$

Alice proposes a simplification where the update gate is removed ($\Gamma_u = 0$).

Betty proposes a simplification where the relevance gate is removed ($\Gamma_r = 1$).

Which model is more robust to the vanishing gradient problem while remaining functional for sequence learning?

- ☒ Betty's model (removing Γ_r), because it retains the update gate Γ_u , allowing the model to decide when to update or keep the memory state.
- ☐ Alice's model (removing Γ_u), because it guarantees that the memory cell state is never overwritten by new, potentially noisy information.
- ☐ Alice's model (removing Γ_u), because simpler models differ less from standard RNNs and thus train more stably.
- ☐ Betty's model (removing Γ_r), because the relevance gate Γ_r handles the short-term dependencies, while the remaining update gate handles long-term dependencies.

✔ Nice work

Yes. This simplification essentially reduces the candidate calculation to a standard RNN update but keeps the gating mechanism. By retaining Γ_u , the network can learn to propagate gradients far back in time by keeping the gate closed (close to 0).

time by keeping the gate closed (close to 0).

9. True/False: Using the equations for the GRU and LSTM below the Update Gate and Forget Gate in the LSTM play a role similar to $1 - \Gamma_u$ and Γ_u .

1 / 1 point

GRU

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$

☒ False

☐ True

✔ Nice work

Instead of using Γ_u to compute $1 - \Gamma_u$, LSTM uses 2 gates (Γ_u and Γ_f) to compute the final value of the hidden state. So, Γ_f is used instead of $1 - \Gamma_u$.

10. True/False: You would use unidirectional RNN if you were building a model map to show how your mood is heavily dependent on the current and past few days' weather.

1 / 1 point

☐ False

☐ True

$$\Gamma_u = \sigma(W_u [c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r [c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

$$\Gamma_u = \sigma(W_u [a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f [a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o [a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$

☒ False

☐ True

✔ Nice work

Instead of using Γ_u to compute $1 - \Gamma_u$, LSTM uses 2 gates (Γ_u and Γ_f) to compute the final value of the hidden state. So, Γ_f is used instead of $1 - \Gamma_u$.

10. True/False: You would use unidirectional RNN if you were building a model map to show how your mood is heavily dependent on the current and past few days' weather.

1 / 1 point

☐ False

☒ True

✔ Nice work

Your mood is contingent on the current and past few days' weather, not on the current, past, AND future days' weather.